

keyestudio - Smart Beach Resort

CHRISTOPH ASTLINGER (2410781003@hochschule-burgenland.at), WOLFGANG GUNDACKER (2410781022@hochschule-burgenland.at), LISA VAN DEN HEUVEL (2410781017@hochschule-burgenland.at), and ALEXANDER PRANTL (2410781004@hochschule-burgenland.at)

CONTENTS

Contents	1
1 Einleitung	2
2 Use Case View	3
2.1 Einzel-Use Cases	3
2.2 Gesamt-Use Case – Smart Beach Resort Management	4
3 Logical View	4
4 Development View	5
5 Process View	7
5.1 Beispielprozess – Sturmwarnung	7
5.2 Kommunikationsparameter	7
5.3 Beispielprozess – Gästewechsel	7
6 Physical View	9
7 Schlussfolgerungen	10
References	12

1 Einleitung

Das Projekt *Smart Beach Resort* untersucht, wie moderne IoT-Technologien in einem praxisnahen Umfeld einsetzbar sind, um den Betrieb von Ferienhäusern effizienter, sicherer und nachhaltiger zu gestalten. An der niederländischen Nordseeküste stehen vier intelligente Modellhäuser, die gemeinsam ein digitales Resort bilden. Die Häuser sind durch ein hybrides Kommunikationssystem miteinander verbunden, das lokale Bluetooth-Verbindungen mit einer Cloud-basierten MQTT-Infrastruktur kombiniert. Jedes Haus erfüllt eine spezifische Funktion und trägt zum Gesamtsystem bei: Überwachung von Wetter und Sturm, Gäste-Komfort und Sicherheit sowie Energie- und Wartungsmanagement. Die Verbindung dieser Einzelfunktionen schafft ein autonom agierendes System, das auf Umweltbedingungen und Nutzeraktionen selbstständig reagiert.

Die physische Umgebung stellt besondere technische Herausforderungen dar. Salzhaltige Luft, hohe Luftfeuchtigkeit, Sandablagerungen und häufige Stürme führen zu Korrosionsgefahr, Sensorbelastung und instabilen Funkbedingungen. Ziel des Projekts ist daher nicht nur die Entwicklung funktionaler Einzellösungen, sondern auch der Nachweis, dass ein IoT-System unter realistischen Umweltbedingungen zuverlässig arbeiten kann. Darüber hinaus soll gezeigt werden, wie drei getrennt entwickelte Einheiten zu einem konsistenten Gesamtsystem verknüpft werden können, das sowohl lokal als auch cloudbasiert funktioniert.

Der Ansatz basiert auf der Nutzung standardisierter IoT-Komponenten. Die Häuser sind mit einem **ESP32**-Controller ausgestattet, der Sensordaten über Temperatur, Luftfeuchtigkeit, Bewegung, Gasgehalt oder RFID-basierte Zugänge erfasst. Die Daten werden über das **MQTT-Protokoll** an den Cloud-Broker **HiveMQ** übertragen und dort verarbeitet. Ein zentraler **Node-RED**-Server übernimmt die Steuerungslogik, aggregiert Ereignisse und reagiert auf erkannte Zustände, beispielsweise durch das Schließen von Fenstern bei Sturm oder das Abschalten der Lüftung bei hoher Feuchtigkeit. Damit entsteht eine klare funktionale Trennung zwischen Geräte-, Kommunikations- und Steuerungsebene.

Die Entwicklung erfolgt dezentral. Jede Teilgruppe arbeitet an einem eigenen Hausmodul, das unabhängig funktionsfähig ist, sich jedoch über ein gemeinsames Topic-System in die Resort-Architektur integrieren lässt. Da die Entwickler an unterschiedlichen Standorten arbeiten, wird die Verbindung über mobile Hotspots realisiert. Dies zeigt, dass auch verteilte Entwicklungsteams IoT-Systeme aufbauen können, ohne auf lokale Netzwerke oder physische Kopplung angewiesen zu sein. Die Cloud-Anbindung erlaubt die Zusammenarbeit in Echtzeit und vereinfacht die Integration der einzelnen Module zu einem gemeinsamen Use Case.

Das Projekt richtet sich an Betreiber von Ferienanlagen, kleine Hotels und Gebäudeverwalter, die ihre Objekte aus der Ferne überwachen und steuern möchten. Die Kombination aus kostengünstiger Mikrocontroller-Hardware, Cloud-Integration und visueller Steuerungslogik (Node-RED) zeigt, dass sich ein voll funktionsfähiges Smart-Home-System auch ohne teure Speziallösungen realisieren lässt. Der Lösungsansatz kann auf andere Einsatzgebiete wie Studentenwohnheime, Serviced Apartments oder modulare Ferienparks übertragen werden. Wirtschaftlich betrachtet bietet das System eine Reduktion von Personalaufwand und Energieverbrauch sowie eine Erhöhung der Betriebssicherheit und Gästezufriedenheit.

Mehrwert:

- Automatisierte Sturm- und Sicherheitswarnungen für das gesamte Resort
- Energieeffizientes Gebäudemanagement durch bedarfsorientierte Steuerung
- Digitale Check-in/Check-out-Prozesse über RFID-Zugangssysteme
- Vereinfachte Wartungskoordination und Zustandsüberwachung in Echtzeit
- Skalierbarkeit auf größere Ferienanlagen durch Cloud-Integration

Insgesamt zeigt das *Smart Beach Resort*-Projekt, dass ein durchdachtes Zusammenspiel von Sensorik, Cloud-Infrastruktur und Automatisierungslogik eine robuste IoT-Architektur ermöglicht, die sowohl für die Forschung als auch für den praktischen Einsatz in der Gebäudeverwaltung relevant ist. Es dient als Blaupause für verteilte Smart-Home-Systeme und demonstriert, dass Cloud-fähige IoT-Frameworks wie HiveMQ und Node-RED die Grundlage für zukunftsfähige, modulare und wartbare Smart-Environment-Lösungen bilden.

2 Use Case View

2.1 Einzel-Use Cases

Haus 1 – Wetter & Perimeterschutz

Haus 1 dient als zentrales Warnsystem. Es misst Temperatur, Luftfeuchtigkeit und Bewegung. Bei hoher Feuchte oder einer simulierten Sturmwarnung aktiviert es den **Sturmmodus** – alle Häuser schließen Fenster, stoppen Lüftung und schalten Warnsignale. Das LCD-Display zeigt aktuelle Wetterdaten, der Buzzer gibt akustische Hinweise. Sensoren: Temperatur-, Feuchte-, Dampf-, Gas-, Bewegungsmelder.

Aktoren: Buzzer, LCD-Display.

Funktion: Frühwarnung bei Sturm oder Gas, Korrosionsschutz, Sicherheitsalarm.

Haus 2 – Gäste-Komfort & Sicherheit

Sensorik/Aktorik: RFID, Buttons, LCD, RGB-LED, Servo, Yellow-LED.

Funktion: kontaktloser Zugang, Service-Anfragen, automatische Rollos, Beleuchtungssteuerung.

Hier liegt der Fokus auf dem Aufenthalt. Ein **RFID-Sensor** ersetzt den Schlüssel, Buttons ermöglichen Check-out oder Serviceanfragen. Ein **Servo** simuliert Rollos oder Türen, während eine **RGB-LED** den Belegungsstatus signalisiert. Das System bildet so einen automatisierten, kontaktlosen Gästeprozess ab – technisch plausibel und übertragbar auf reale Vermietungssysteme.

Haus 3 – Partyhaus

Im Gesamtszenario des Smart Beach Resort ergänzt Haus 3 die Funktionen von Haus 2, das den Gästezugang und den Check-in-Prozess automatisiert, um einen Use Case, der den Aufenthalt der Gäste durch gezielte Licht- und Audioinszenierungen erweitert. Haus 3 dient als Demonstrationsumgebung für eine ereignisgesteuerte Entertainment-Steuerung innerhalb des Resorts und zeigt, wie IoT-Knoten neben sicherheits- und verwaltungsorientierten Aufgaben auch erlebnisorientierte Dienste bereitstellen können.

Der Einzel-Use-Case basiert auf der Verwendung des im KS5009-Kit enthaltenen RGB-LED-Moduls sowie des passiven Buzzers, die gemeinsam eine abgestimmte Sequenz aus Licht- und Toneffekten erzeugen. Die Aktivierung dieser Szene erfolgt über das zentrale Node-RED-Dashboard, das dem Betreiber die Buchung einer „Party“ ermöglicht. Während eine solche Party aktiv ist, können Gäste außerdem mittels der im Projekt vorhandenen RFID-Chips unterschiedliche Musikstücke abrufen, wodurch das System eine Art „Tonibox für junggebliebene Urlauber“ abbildet und interaktiv einige Musikstücke in die Entertainment-Logik integriert. Das LCD-Display des Hauses zeigt parallel den aktuellen Szenenstatus wie „Party läuft“ oder den Zeitpunkt der nächsten, geplanten Feier und dient so als lokale Rückmeldung über den Betriebsmodus.

Aus architektonischer Sicht demonstriert Haus 3 damit die Service-Schicht innerhalb des Gesamtsystems. Während andere Häuser sicherheits- oder umweltbezogene Aufgaben übernehmen, repräsentiert dieses Haus den Aspekt der User Experience im IoT-Kontext. Die Implementierung zeigt, dass verteilte Mikrocontroller nicht nur Messdaten erfassen, sondern auch ereignisgesteuert auf Benutzeranforderungen reagieren können.

Christoph Astlinger (2410781003@hochschule-burgenland.at), Wolfgang Gundacker (2410781022@hochschule-burgenland.at), Lisa van den Heuvel (2410781017@hochschule-burgenland.at),
4 and Alexander Prantl (2410781004@hochschule-burgenland.at)

Da bei einer Party laute Musik eine gängige Begleiterscheinung darstellt und die Häuser ein gemeinsames Resort bilden, wird beim Start des Partymodus ein Schutzmechanismus aktiviert: Die Fenster und Türen der übrigen Häuser werden automatisch geschlossen, um mögliche Lärmbelastigungen für andere Gäste zu minimieren.

Haus 4 – Luftqualität & Automatische Lüftung

Haus 4 sorgt für Sicherheit und Komfort. Ein **Gas-Sensor** überwacht CO- oder H₂-Konzentrationen. Bei Überschreitung startet automatisch der **Ventilator**, das Dashboard zeigt eine Warnung, und ein Signalton informiert den Betreiber. Sinken die Werte wieder, schaltet das System selbstständig zurück in den Normalbetrieb. Ein **Button** dient zur manuellen Rücksetzung.

2.2 Gesamt-Use Case – Smart Beach Resort Management

Alle drei Häuser bilden ein gemeinsames IoT-System:

- Zentrales Dashboard (Node-RED) zur Gesamtsteuerung
- Automatische Fenster- und Lüftungssteuerung bei Sturm
- Synchronisierte Reinigungs- und Energieprozesse
- Internet-Anbindung über Handy-Hotspots



to be done!

Fig. 1. Use-Case-Diagramm mit Akteuren (Gast, Host, Reinigungspersonal)

3 Logical View

Im Zentrum des Systems stehen vier logisch getrennte, jedoch über MQTT vernetzte Komponenten: das Wetter- und Sicherheitshaus, das Gästehaus, das Partyhaus und das Luftqualitätsmodul. Jedes dieser Häuser bildet einen eigenständigen logischen Knoten, der auf lokale Ereignisse reagiert und zugleich globale Systemzustände berücksichtigt.

Die wesentlichen Funktionen werden durch Sensorzustände ausgelöst, durch interne Logik verarbeitet und in Form von Aktorbefehlen manifestiert.

Die Kernfunktion von Haus 1 liegt in der Überwachung des Umweltzustands. Die logische Komponente „Wetteranalyse“ berechnet aus Temperatur-, Feuchte- und Dampfsensorwerten, ob eine meteorologische Warnsituation vorliegt. Wird ein kritischer Zustand erkannt oder im Dashboard ein Sturmereignis ausgelöst, wechselt das System in den globalen Zustand „Sturmmodus“. Dieser Zustand wird über MQTT verbreitet und beeinflusst unmittelbar den Ablauf in allen anderen Häusern. Parallel dazu verwaltet Haus 1 eine „Perimeterschutz“-Logik, die bei erkannter Bewegung im Leerstand den Sicherheitszustand erhöht und eine optische Rückmeldung erzeugt.

Haus 2 bildet die logische Einheit für Zugangskontrolle und Nutzungsstatus. RFID-Authentifizierung führt zu einem Zustandswechsel von „leer“ zu „belegt“, der intern als Ausgangspunkt für Komfortfunktionen genutzt wird. Ein Checkout ruft den umgekehrten Übergang hervor. Die Zustandsmaschine dieses Hauses koppelt den Belegungsstatus an die Aktorlogik, sodass beispielsweise Servobewegungen oder Lichtsignale direkt aus dem internen Hauszustand abgeleitet werden. Gleichzeitig reagiert das System auf globale Ereignisse wie den Sturmmodus, der unabhängig von der Belegung zu einem sicheren Zustand führt.

Haus 3 erweitert die logische Gesamtsicht um eine ereignisbasierte Entertainmenteinheit. Die Steuerlogik unterscheidet zwischen den Zuständen „Party aktiv“ und „Party inaktiv“. Der Übergang in den aktiven Zustand kann sowohl über das Dashboard als auch über RFID-Interaktionen erfolgen, die im Kontext der Party unterschiedliche Melodien oder Szenen anstoßen. Die Licht- und Tonlogik folgt dabei einer internen Sequenzsteuerung, die auf dem ESP32 ausgeführt wird. Wird ein globaler Sturmmodus ausgelöst oder eine Party manuell beendet, wechselt die Komponente unverzüglich in den sicheren Zustand, stoppt alle Effekte und synchronisiert diesen Übergang über MQTT.

Haus 4 stellt die logische Komponente zur Überwachung und Regulierung der Luftqualität dar. Der Systemzustand „Luftqualität kritisch“ wird aus dem Gas-Sensorwert abgeleitet und triggert unmittelbar den Übergang zu „Lüftung aktiv“. Die Aktorsteuerung ist an eine interne Rückführlogik gekoppelt, die selbständig beendet wird, sobald die gemessenen Werte die definierte Normzone wieder erreichen. Ein manueller Reset kann situativ erfolgen und führt zum Rücksprung in den Ausgangszustand. Auch dieses Haus berücksichtigt den globalen Sturmmodus, der die Lüftung aus Sicherheitsgründen übersteuert.

Die globale logische Sicht ist damit durch zwei Ebenen charakterisiert: durch lokale Zustandsmaschinen in jedem Haus sowie durch geteilte Systemzustände, die über MQTT synchronisiert werden. Das Zusammenspiel dieser Ebenen ermöglicht ereignisorientierte Abläufe, bei denen sowohl autonome Hauslogik als auch zentrale Steuerkommandos konsistent berücksichtigt werden. Dadurch entsteht ein kohärentes Verhalten des gesamten Resorts, das unabhängig vom physischen Aufbau als strukturierte, verteilte Logik modelliert und nachvollziehbar beschrieben werden kann.

4 Development View

Das Gesamtsystem besteht aus zwei klar getrennten Entwicklungsdomänen: der eingebetteten Software auf den ESP32-basierten Häusern und der Steuerungslogik innerhalb von Node-RED auf dem Resort-Server. Diese Trennung spiegelt sowohl den technischen Aufbau als auch die Verantwortung der Komponenten wider. Die Software der vier Häuser ist modular aufgebaut, wobei alle ESP32-Controller ein identisches Grundgerüst nutzen, das aus drei logischen Paketen besteht: einem Hardware-Abstraktionsmodul, einem MQTT-Kommunikationsmodul und einem hausindividuellen Service-Modul. Die Hardware-Abstraktion kapselt sämtliche Sensor- und Aktorzugriffe in wiederverwendbare Funktionen, sodass die Applikationslogik nicht direkt mit GPIO-Konfigurationen oder Gerätespezifika interagiert. Das

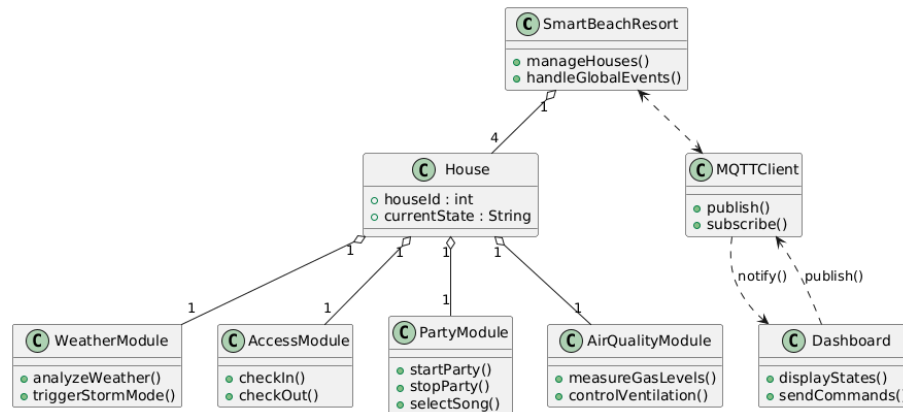


Fig. 2. Logical View/Klassen

Kommunikationsmodul implementiert die Anbindung an die HiveMQ Cloud über MQTT und stellt publish/subscribe-Schnittstellen bereit. Die eigentlichen fachlichen Verhaltensweisen der Häuser – Wetteranalyse, Zugangskontrolle, Partylogik und Luftqualitätsmanagement – befinden sich in den hausindividuellen Service-Modulen, die auf die beiden Basispakete zugreifen.

Node-RED bildet die zweite Entwicklungsdomäne und ist als visuelles Integrations- und Prozessautomationssystem strukturiert. Die Node-RED-Implementierung besteht aus mehreren Flows, die in logisch getrennte Funktionsbereiche unterteilt sind. Ein Flow ist für die Entgegennahme aller MQTT-Nachrichten zuständig und übersetzt sie in semantische Systemereignisse. Ein weiterer Flow verwaltet globale Zustände, etwa den Sturmmodus oder synchronisierte Aktionen über mehrere Häuser hinweg. Zusätzlich existieren dedizierte Haus-Flows, die UI-Elemente, Dashboard-Ausgaben und Servicefunktionen bündeln. Da Node-RED in einem Docker-Container betrieben wird, lässt sich die gesamte Entwicklungsumgebung reproduzierbar versionieren und isolieren. Der Quelltext der Flows wird in einer JSON-Repräsentation gespeichert, wodurch Änderungen nachverfolgbar bleiben und in Git verwaltet werden können.

Die Zusammenarbeit zwischen ESP32-Software und Node-RED basiert auf klar definierten MQTT-Themenstrukturen. Dadurch entsteht kein Kopplungsbedarf zwischen Entwicklungsteams oder Geräten, sondern eine lose, nachrichtenbasierte Architektur. Änderungen an einem Haus betreffen nur dessen Service-Modul, solange die veröffentlichten Themen und Nachrichtenformate unverändert bleiben. Diese Entkopplung erleichtert sowohl Wartung als auch Erweiterung: Ein zusätzliches Haus kann eingebunden werden, indem lediglich ein weiteres Service-Modul auf Basis der bestehenden Grundkomponenten erstellt und an die MQTT-Struktur angebunden wird.

Aus Entwicklersicht ist die Architektur so aufgebaut, dass Verantwortlichkeiten sichtbar voneinander getrennt sind. Hardware-Nähe, Kommunikationsinfrastruktur und fachliche Logik sind in getrennte Module gefasst, während Node-RED die Rolle eines zentralen Integrationslayers einnimmt. Diese Modularisierung erleichtert Fehlerlokalisierung, erlaubt parallele Entwicklung und reduziert Konflikte zwischen Funktionsbereichen. Durch die Nutzung eines Git-Repositories können sowohl die Firmware der Häuser als auch die Node-RED-Flows versioniert werden, wodurch eine klare Trennung zwischen stabilen Releases und experimentellen Entwicklungsständen möglich wird.

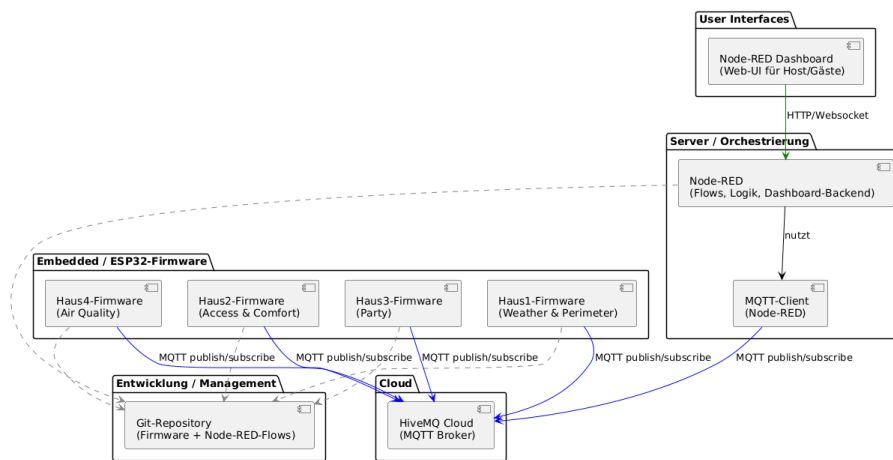


Fig. 3. Komponentendiagramm

5 Process View

5.1 Beispielprozess – Sturmwarnung

- (1) Haus 1 erkennt erhöhte Feuchtigkeit und Temperaturabfall.
- (2) MQTT-Nachricht `resort/all/cmd/storm_alert=true`.
- (3) Node-RED sendet Befehle:
 - `resort/house2/cmd/shutters=DOWN`
 - `resort/house3/cmd/ventilation=OFF`
- (4) Häuser bestätigen Status über `resort/house/status`.

5.2 Kommunikationsparameter

- Keep-Alive 60 s, Heartbeat 30 s
- QoS 1 für kritische Events
- Nachrichtengröße ca. 150 Byte
- Datenrate ca. 6–12 kB pro Sensor und Stunde

5.3 Beispielprozess – Gästewechsel

- (1) RFID-Check-out → Event checkout.
- (2) Node-RED startet Reinigungstimer für Haus 3.
- (3) Reinigung meldet Button-Event `cleaned`.
- (4) Node-RED reaktiviert Lüftung und bereitet Check-in vor.

Zur Laufzeit lässt sich das System grob in drei Prozessklassen unterteilen: die vier eingebetteten Steuerungen der Häuser, den MQTT-Broker in der Cloud und den Node-RED-Prozess als Koordinationsinstanz auf dem Resort-Server. Jedes Haus wird durch einen eigenständigen ESP32-Mikrocontroller betrieben, der seine lokale Logik in einem ereignis-gesteuerten Hauptloop ausführt. In diesem Loop werden zyklisch Sensordaten erfasst, Zustände ausgewertet und auslösende Bedingungen für Aktoren oder Nachrichten geprüft. Zusätzlich verarbeitet jeder Controller asynchrone



to be done!

Fig. 4. Sequenzdiagramm für Sturm- und Check-out-Prozesse

Ereignisse aus der MQTT-Kommunikation, etwa Befehle aus Node-RED oder globale Systemzustände wie den Sturmmodus. Aus Sicht der Process View handelt es sich bei den Häusern also um vier parallel ablaufende, voneinander unabhängige Steuerprozesse, die ausschließlich über den Broker gekoppelt sind.

Der MQTT-Broker in der HiveMQ Cloud bildet zur Laufzeit den zentralen Vermittlungsprozess. Er verwaltet die Verbindungen aller Häuser und von Node-RED, nimmt Nachrichten entgegen und verteilt sie an alle Clients, die die entsprechenden Topics abonniert haben. Der Broker selbst enthält keine Fachlogik des Resorts, spielt aber eine zentrale Rolle in der Prozesssicht, da er die zeitliche Entkopplung der Komponenten ermöglicht. Ein Haus kann zu einem beliebigen Zeitpunkt eine Nachricht publizieren, ohne zu wissen, ob Node-RED sie bereits verarbeitet oder gerade ausgelastet ist. Umgekehrt kann Node-RED Kommandos senden, die von den Häusern bei der nächsten verfügbaren Kommunikationsschleife abgeholt werden. Diese lose Kopplung ist charakteristisch für die Prozessarchitektur des Systems.

Node-RED läuft auf dem Server in einem eigenen Prozess, der als Einstiegspunkt für alle höheren Steuerungs- und Orchestrierungsaufgaben fungiert. Innerhalb von Node-RED sind die Funktionen in Flows organisiert, die sich zur Laufzeit wie leichtgewichtige Teilprozesse verhalten. Ein Flow empfängt beispielsweise alle eingehenden MQTT-Nachrichten, normalisiert sie und verteilt sie an weitere Flows, die jeweils für einen Hauskontext oder einen Systemkontext zuständig sind. Ein anderer Flow verwaltet globale Zustände wie den Sturmmodus, indem er eingehende Ereignisse analysiert, interne Zustandsvariablen anpasst und wiederum MQTT-Nachrichten an die Häuser publiziert. Parallel dazu laufen die Dashboard-Flows, die HTTP- oder WebSocket-Anfragen der Benutzer verarbeiten, UI-Ereignisse in MQTT-Kommandos übersetzen und Zustandsänderungen in der Benutzeroberfläche widerspiegeln. Aus Sicht der Process View bildet Node-RED damit einen Multi-Flow-Prozess, der mehrere logische Kontrollstränge parallel abbildet, auch wenn sie in einem technischen Prozessraum ausgeführt werden.

Das Zusammenspiel dieser Prozesse lässt sich exemplarisch am Ablauf des Sturmmodus illustrieren. Im Normalbetrieb erfassen die Sensorprozesse in Haus 1 in regelmäßigen Intervallen Temperatur und Feuchtigkeit und prüfen, ob definierte Schwellenwerte überschritten werden oder ob über das Dashboard eine Sturmwarnung aktiviert wurde. Wird eine kritische Situation erkannt, publiziert die Firmware von Haus 1 eine entsprechende MQTT-Nachricht auf einem vordefinierten Topic. Der Broker nimmt diese Nachricht entgegen und leitet sie an Node-RED weiter, wo ein Flow diese Ereignisnachricht verarbeitet und den globalen Zustand „Sturmmodus aktiv“ setzt. In Folge generiert derselbe Flow neue MQTT-Nachrichten, die an alle Häuser adressiert sind. In den Prozessen von Haus 2, 3 und 4 werden diese Kommandos beim nächsten Kommunikationszyklus empfangen und führen dort zu lokalen Zustandsübergängen: Servos schließen Fenster oder Rollos, laufende Partys werden beendet, Lüftungen werden deaktiviert oder in einen Sicherheitsmodus versetzt. Parallel aktualisiert ein Dashboard-Flow die Visualisierung für den Host und zeigt den aktiven Sturmmodus sowie die Reaktionen der Häuser an. Damit wird deutlich, dass mehrere Prozesse, verteilt über Endgeräte, Cloud und Server, in einem koordinierten, ereignisorientierten Ablauf zusammenwirken.

Ein zweites Beispiel ist der Ablauf einer Party in Haus 3. Hier löst entweder der Benutzer am Dashboard oder ein lokales RFID-Ereignis im ESP32-Prozess von Haus 3 die Aktivierung des Partyzustands aus. Im ersten Fall sendet Node-RED eine MQTT-Nachricht an Haus 3, im zweiten Fall publiziert Haus 3 selbst einen Zustandswechsel an den Broker, der in Node-RED zur Aktualisierung der UI führt. In beiden Fällen läuft auf dem Controller von Haus 3 ein separater Logikpfad an, der die Sequenzsteuerung von Licht- und Tonmustern übernimmt und in kurzen Intervallen Buzzer- und RGB-LED-Aktionen triggert. Gleichzeitig bleibt der Prozess in der Lage, weitere eingehende MQTT-Nachrichten zu verarbeiten, etwa das Deaktivieren der Party oder den Eintritt in den Sturmmodus. Prozessual betrachtet handelt es sich um einen lokal priorisierten Effektprozess, der jederzeit von globalen Ereignissen übersteuert werden kann.

Auch der Luftqualitätsprozess in Haus 4 folgt diesem Muster. Ein zyklischer Messprozess prüft die Gaswerte und löst bei Überschreitung einer Schwelle unmittelbar die Aktivierung des Lüfters und die lokale Alarmierung aus. Parallel sendet der ESP32 eine Alarmpublikation an den Broker, die in Node-RED wiederum einen Flow zur Protokollierung und Anzeige auf dem Dashboard auslöst. Sobald die Luftqualität wieder unkritisch ist, wird im Hausprozess die Lüftung deaktiviert und ein Rückmeldesignal publiziert. Auch hier sind mindestens drei Prozesse beteiligt: die lokale Mess- und Regelungsschleife, der Broker als Vermittler und Node-RED als Auswerter und Visualisierer des Ereignisses.

In der Gesamtschau zeigt die Process View des Smart Beach Resort, dass es sich um ein verteiltes, ereignisgesteuertes System handelt, in dem mehrere parallel laufende Prozesse über einen Nachrichtenbus interagieren. Die Häuser führen ihre lokalen Steuerungsaufgaben autonom aus, Node-RED koordiniert globale Abläufe und präsentiert den Systemzustand, und der MQTT-Broker entkoppelt zeitlich und räumlich alle beteiligten Einheiten. Diese Prozessarchitektur sorgt dafür, dass das System skalierbar bleibt und neue Häuser oder Funktionen durch Hinzufügen weiterer, parallel laufender Komponenten integriert werden können, ohne die bestehenden Abläufe grundlegend verändern zu müssen.

6 Physical View

Hardware-Mapping:

10

Komponente	Beschreibung
ESP32 DevBoard	Controller mit WiFi und Bluetooth
Sensoren	DHT11, Steam, Gas, PIR, RFID, Buttons
Aktoren	Buzzer, Servo, LED, Lüfter
Gateway	Smartphone-Hotspot pro Haus
Broker	HiveMQ Cloud (Serverless)
Controller	Node-RED auf privatem Server

Netzstruktur:

- Jedes Haus → eigenes IoT-Node mit Hotspot-Anbindung.
- Kommunikation über MQTT zu HiveMQ Cloud.
- Node-RED abonniert resort/# für zentrale Steuerung.



to be done!

Fig. 5. Deployment-Diagramm (ESP32 – HiveMQ – Node-RED – App)

7 Schlussfolgerungen

Das Projekt zeigt, dass ein verteiltes IoT-System mit mobilen Hotspots zuverlässig funktioniert, wenn ein Cloud-basiertes Kommunikationsframework eingesetzt wird. HiveMQ + Node-RED bietet eine modulare, skalierbare Architektur.

Ergebnisse:

- Sichere, stabile Kommunikation über MQTT Cloud-Broker
- Automatisierte Prozesssteuerung per Node-RED
- Cloud- und On-Premise-Betrieb möglich
- Dezentrale Entwicklung ohne physische Vernetzung

Kosten:

- Hardware: Anschaffung ca. 210 € (3 × KS5009-Kits)
- Cloud-Infrastruktur: 0 €/Monat bzw. Serverkosten bei Neuanmietung

Key Take-Aways:

- MQTT ist das stabilste Kommunikationsprotokoll für IoT.
- Cloud-Broker beseitigen NAT-Probleme bei Hotspots.
- Node-RED vereinfacht Regellogik und Visualisierung.
- System ist erweiterbar und demofähig unter realen Bedingungen.

Christoph Astlinger (2410781003@hochschule-burgenland.at), Wolfgang Gundacker
(2410781022@hochschule-burgenland.at), Lisa van den Heuvel (2410781017@hochschule-burgenland.at),
and Alexander Prantl (2410781004@hochschule-burgenland.at)

12

References